Vision Transformers for Image Classification

1st Arthur Gubaidullin
B22-DS-01
Innopolis University
Innopolis, Russian Federation
a.gubaidullin@innopolis.university

2nd Leonid Novikov
B22-RO-01
Innopolis University
Innopolis, Russian Federation
l.novikov@innopolis.university

3rd Elina Murtazina
B22-RO-01
Innopolis University
Innopolis, Russian Federation
el.murtazina@innopolis.university

***Abstract*—In this paper we will implement relatively new approach for Image Classification - Vision Transformers architecture.**

I. Introduction/Related Work

Over the last decade Convolutional Neural Networks (CNNs) were leading in Image Classification tasks due to their ability to capture important information from the data.

However, recently scientists started to implement Vision Trasformers (ViT) [1]. Our main goal is to implement ViT for an Image classification task.

The main differences between original Transformer [2] and Vit are presented in Fig. 1

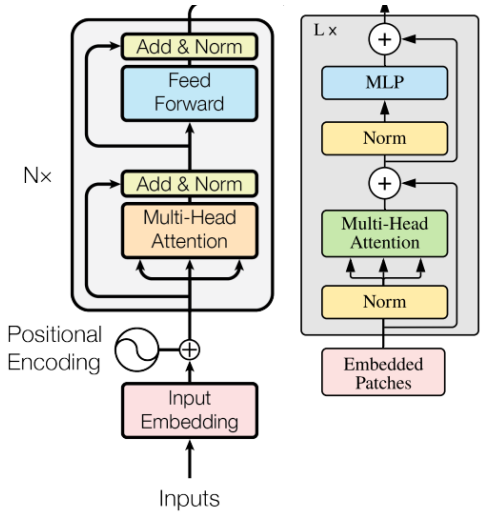

Fig. 1. Original Transformer Encoder (on the left) and Vit Encoder (on the right)

- **Input Representation:**
  - **Transformer Encoder:** Processes a sequence of token embeddings.
  - **ViT Encoder:** Processes a sequence of patch embeddings derived from image data.
- **Positional Encoding:**
  - **Transformer Encoder:** Uses positional encodings to inject sequence order information.
  - **ViT Encoder:** Uses positional encodings to inject spatial information of image patches.
- **Input Embedding:**
  - **Transformer Encoder:** Directly uses token embeddings.
  - **ViT Encoder:** Uses a linear projection of flattened image patches to generate patch embeddings.

II. Technique/Method

Our model takes an image, splits it into small square patches, and flattens them so the ViT can process them like a sequence of words. Each patch is transformed into a fixed-size vector, called a "patch embedding".

We also add a special token at the start to represent the whole image, similar to how Large Language Model (LLM) "BERT" handles sentences. During training, we attach a classifier to this token to predict the image's class.

To help the model understand where each patch belongs, we add position information to each patch. The resulting sequence is then passed through the VT.

A. Patch Emgeddings

Our initial implementation of Patch Embeddings class uses linear projection to generate embeddings:

1) **Input Tensor:** The input tensor is represented as:

$$\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W},$$

where:

- B: Batch size.
- C: Number of channels (e.g., 3 for RGB images).
- H: Height of the input image.
- W: Width of the input image.

2) **Patch Extraction:** The input image is divided into $N = \frac{H \cdot W}{P^2}$ non-overlapping patches, where P is the patch size. Each patch is flattened into a vector:

$$\mathbf{P}_{i,j} = \text{Flatten}(\mathbf{X}_{i,j}) \in \mathbb{R}^{P^2 \cdot C}, \quad \forall i, j,$$

resulting in the following tensor:

$$\mathbf{P} \in \mathbb{R}^{B \times N \times (P^2 \cdot C)}.$$

3) **Linear Projection:** Each patch is projected into an embedding space of dimension D using a learnable linear transformation:

$$\mathbf{Z} = \mathbf{P}\mathbf{W} + \mathbf{b},$$

where:

- $\mathbf{W} \in \mathbb{R}^{(P^2 \cdot C) \times D}$: Linear projection weights.
- $\mathbf{b} \in \mathbb{R}^{D}$: Bias term.

The resulting tensor has dimensions:

$$\mathbf{Z} \in \mathbb{R}^{B \times N \times D}.$$

4) **Class Token:** A learnable class embedding $\mathbf{z}_{\text{class}} \in \mathbb{R}^{B \times 1 \times D}$ is prepended to the sequence of patch embeddings:

$$\mathbf{Z}_{\text{class}} = [\mathbf{z}_{\text{class}}; \mathbf{Z}],$$

giving the tensor:

$$\mathbf{Z}_{\text{class}} \in \mathbb{R}^{B \times (N+1) \times D}.$$

5) **Positional Embeddings:** A learnable positional embedding $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{B \times (N+1) \times D}$ is added to the patch embeddings:

$$\mathbf{Z}_{\text{final}} = \mathbf{Z}_{\text{class}} + \mathbf{E}_{\text{pos}},$$

where:

- $\mathbf{E}_{\text{pos}}$ is broadcasted along the batch dimension and repeats for all B images.

The final output of the PatchEmbedding layer is a tensor:

$$\mathbf{Z}_{\text{final}} \in \mathbb{R}^{B \times (N+1) \times D},$$

where:

- B: Batch size.
- $N = \frac{H \cdot W}{P^2}$: Number of patches.
- D: Embedding dimension.

The following constraint must hold for the input image dimensions:

$$H \mod P = 0 \quad \text{and} \quad W \mod P = 0,$$

ensuring that the image can be evenly divided into patches of size $P \times P$.

B. Patch Embeddings V2

Additionally we implemented alternative approach using torch.nn.Conv2D:

1) **Input Tensor:** The input tensor is represented as:

$$\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W},$$

where:

- B: Batch size.
- C: Number of color channels.
- H: Height of the input image.
- W: Width of the input image.

2) **Patch Partitioning:** The input image is divided into non-overlapping patches of size $P \times P$, where P is the *patch size*. This is implemented using a 2D convolution operation:

$$\mathbf{X}_{\text{patched}} = \text{Conv2D}(\mathbf{X}, P * P, \text{channels out} = D),$$

where:

- D: Embedding dimension (output channels).

The resulting patched tensor has dimensions:

$$\mathbf{X}_{\text{patched}} \in \mathbb{R}^{B \times D \times \frac{H}{P} \times \frac{W}{P}}.$$

3) **Flattening:** Each patch is flattened into a vector, resulting in a sequence of patch embeddings. Mathematically:

$$\mathbf{X}_{\text{flattened}} = \text{Flatten}(\mathbf{X}_{\text{patched}}, \text{start\_dim} = 2, \text{end\_dim} = 3),$$

with dimensions:

$$\mathbf{X}_{\text{flattened}} \in \mathbb{R}^{B \times D \times N},$$

where:

$$N = \frac{H}{P} \times \frac{W}{P}.$$

4) **Permutation:** Finally, the sequence of patch embeddings is transposed to match the input format for a transformer:

$$\mathbf{X}_{\text{output}} = \mathbf{X}_{\text{flattened}}^{\top},$$

giving the output tensor:

$$\mathbf{X}_{\text{output}} \in \mathbb{R}^{B \times N \times D}.$$

The final output of the PatchEmbedding layer is a tensor $\mathbf{X}_{\text{output}}$ representing the patch embeddings for all images in the batch:

$$\mathbf{X}_{\text{output}} \in \mathbb{R}^{B \times N \times D},$$

where:

- B: Batch size.
- $N = \frac{H}{P} \times \frac{W}{P}$: Number of patches.
- D: Embedding dimension.

The following constraint must hold for the input image dimensions:

$$H \mod P = 0 \quad \text{and} \quad W \mod P = 0,$$

ensuring that the image can be evenly divided into patches of size $P \times P$.

C. Encoder Layer

The ViT Encoder operates through a series of transformations, maintaining the same shape throughout the process:

1) **Layer Normalization**: The input tensor $\mathbf{X}$ is normalized using layer normalization:

$$\mathbf{X}_{\text{norm}} = \text{LayerNorm}(\mathbf{X})$$

2) **Multi-Head Self-Attention**: The normalized input $\mathbf{X}_{\text{norm}}$ is passed through a Multi-Head Self-Attention mechanism:

$$\mathbf{Y}_{\text{attn}}, \_ = \text{MultiheadAttention}(\mathbf{X}_{\text{norm}}, \mathbf{X}_{\text{norm}}, \mathbf{X}_{\text{norm}})$$

where $\mathbf{Y}_{\text{attn}}$ is the output of the attention mechanism.

3) **Residual Connection #1**: The output from the attention mechanism is added to the original input $\mathbf{X}$:

$$\mathbf{R}_0 = \mathbf{X} + \mathbf{Y}_{\text{attn}}$$

where $\mathbf{R}_0$ is the first residual connection.

4) **Layer Normalization**: The first residual connection $\mathbf{R}_0$ is normalized using another layer normalization:

$$\mathbf{R}_{\text{norm}} = \text{LayerNorm}(\mathbf{R}_0)$$

5) **Feed-Forward Network (MLP)**: The normalized residual connection $\mathbf{R}_{\text{norm}}$ is passed through a feed-forward network (MLP) consisting of two linear layers with a GELU activation function in between:

$$\mathbf{Z} = \text{Linear}_2(\text{Dropout}(\text{GELU}(\text{Linear}_1(\mathbf{R}_{\text{norm}}))))$$

where $\mathbf{Z}$ is the output of the MLP.

6) **Residual Connection #2**: The output from the MLP is added to the first residual connection $\mathbf{R}_0$:

$$\mathbf{R}_1 = \mathbf{R}_0 + \mathbf{Z}$$

where $\mathbf{R}_1$ is the final output of the Encoder block.

The final output $\mathbf{R}_1$ has the same shape as the input $\mathbf{X}$.

D. Vision Transformer

Our ViT implementation consists of several components, including patch embedding, a stack of encoder blocks, and a final multi-layer perceptron (MLP) for classification:

1) **Patch Embedding**: The input image tensor $\mathbf{X}$ with shape (b, c, h, w) is transformed into patch embeddings. Depending on the specified version (patch_embedding_version), either version 1 or version 2 of the 'PatchEmbedding' class is used:

$$\mathbf{E} = \text{PatchEmbedding}(\mathbf{X})$$

where $\mathbf{E}$ has shape $(b, n+1, n\_model)$ for version 1 and (b, n, n_model) for version 2.

2) **Encoder Stack**: The patch embeddings $\mathbf{E}$ are passed through a stack of num_layers encoder blocks. Each encoder block consists of multi-head self-attention and a feed-forward network (MLP) with residual connections:

$$\mathbf{E}_i = \text{Encoder}(\mathbf{E}_{i-1}) \quad \text{for} \quad i = 1, 2, \ldots, \text{num\_layers}$$

where $\mathbf{E}_0 = \mathbf{E}$ and $\mathbf{E}_i$ is the output of the i-th encoder block.

3) **Class Embedding Extraction**: After passing through all encoder blocks, the class embedding (the first token in the sequence) is extracted:

$$\mathbf{C} = \mathbf{E}_{\text{num\_layers}}[:, 0]$$

where $\mathbf{C}$ has shape (b, n_model).

4) **Classification MLP**: The class embedding $\mathbf{C}$ is passed through a multi-layer perceptron (MLP) for classification. The MLP consists of layer normalization, a linear layer, and another linear layer that maps to the number of classes:

$$\mathbf{R} = \text{MLP}(\mathbf{C})$$

where $\mathbf{R}$ has shape $(b, \text{num\_classes})$.

The final output $\mathbf{R}$ contains the probabilities for each class.

III. Dataset Explanation

Due to Transformer's bad nature of capturing long-term insights and patterns from large datasets, we are planning to use CIFAR-10 [3] dataset. It contains 50 000 training images with 10 classes, 5 000 images for each class. On the other hand, test set contains 10 000 images with 1 000 images per class. Each image has 3 channels and fixed size 32x32.

IV. GitHub repository

Our GitHub repository can be found here.

V. Experiments and Evaluation

Settings and results of all experiments can be found here. For some experiments, images containing the whole training process exist.

A. Important Discovery

As we described earlier, we used 2 versions of Patch Embeddings:

1) Using Linear Projection and Einops;
2) Using torch.nn.Conv2D.

During the training process we used Adam Optimizer and Cross Entropy Loss. We discovered that using the first version of Patch Embeddings training and test loss does not decrease: Fig. 2, Fig. 3

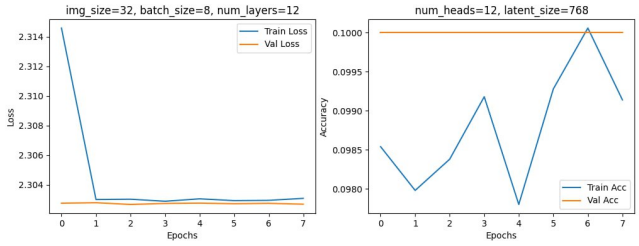

Fig. 2. Loss remains the same throughout the training. Accuracy=0.1 means that our model generates predictions randomly.

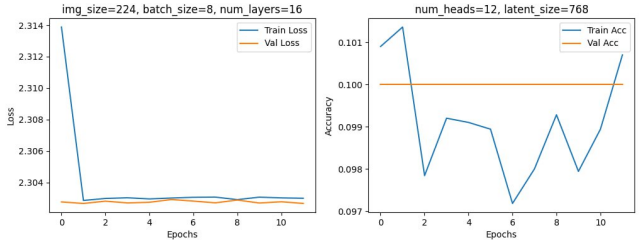

Fig. 3. Same results as on Fig. 2.

After careful investigation of the Encoder and Patch Embedding modules, we found out that Linear Projection layer inside Patch Embedding module had gradients of its weights and biases equal to zero at any point of time. Thus, any change inside Linear Projection layer's parameters does not influence output.

To address this issue, we implemented the second version of Patch Embedding module using torch.nn.Conv2D. Here are the most notable results of training using the second version of Patch Embedding module:

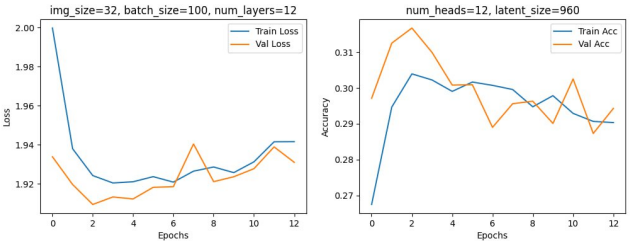

Fig. 4. The notable Loss improvement. However, model still fails to perform well both on training and test datasets.

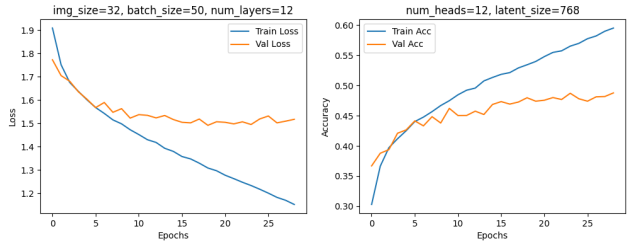

Fig. 5. Model is able to capture information from the data. With each epoch the velocity of loss progression on training dataset remains almost the same, while on test dataset the velocity proceeds to decrease.

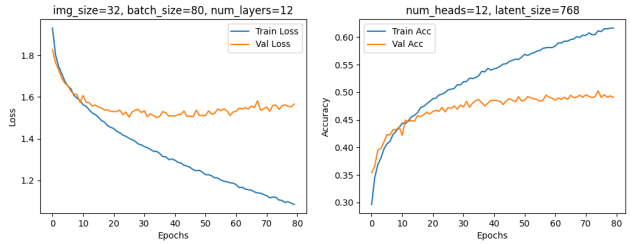

Fig. 6. To investigate more on overfitting signs in Fig. 5, we trained ViT for 80 epochs. The model reached its peak performance on the test dataset with current settings. However, the training Loss proceeds to decrease.

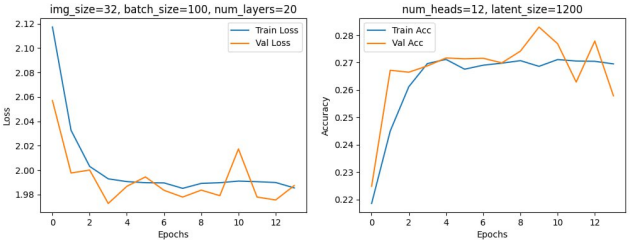

Fig. 7. The model with larger batch size, number of Encoder layers, and latent size. ViT fails to capture important information from the training dataset.

VI. Conclusion

ViTs [1] showed promising results in image classification task, but require a proper setting and hyperparameter tuning.

After training ViT with different hyperparameter settings, we recommend the following settings when training on CIFAR-10 [3] dataset:

- Batch size - [50, 100];
- Number of Encoder layers - [12, 20];
- Number of Heads in the Attention layer - [12, 20];
- Latent size - 768;
- Number of training epochs - [60, 80].

Important to note that ViT do perform well on large datasets, such as ImageNet [4]. From our point of view, the size of CIFAR-10 [3] dataset is not sufficient to achieve state-of-the-art performance in image classification tasks.

References

[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[3] Will Cukierski. Cifar-10 - object recognition in images, 2013.

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database, 2009.